

ML Fundamentals

Core Concepts Every ML Practitioner Must Know

■ FOUNDATIONS

■ LEARNING OUTCOME

By the end of this module you will understand what machine learning is, how machines learn from data, the ML taxonomy, and how to frame any real-world problem as an ML task — the mental model every practitioner needs.

■ Skills You Will Gain

- Define machine learning and distinguish it from traditional programming
- Classify problems as supervised, unsupervised, or reinforcement learning
- Describe the end-to-end ML pipeline in 8 steps
- Explain overfitting, underfitting, and the bias-variance tradeoff
- Perform and explain train/validation/test splits
- Choose the right evaluation metric for a given problem

■ ML engineers earn ■8–40 LPA in India and \$90K–\$180K globally. Every tech company is actively hiring ML talent. This is the decade's defining skill.

SECTION 1

What is Machine Learning?

Machine learning is a branch of AI where systems **learn patterns from data** instead of following hand-coded rules. A spam filter that learns from examples is ML; one built with IF-ELSE rules is not.

Traditional Programming vs Machine Learning

Traditional: Input + Rules → Output (programmer writes every rule) Machine Learning: Input + Output → Rules (model discovers rules from data) Once trained, an ML model acts like a function: new input → prediction output

ML Type	Description	Examples
Supervised	Labeled data: learn input→output mapping	Spam filter, house prices, churn
Unsupervised	No labels: find hidden structure in data	Customer segments, anomaly detection
Semi-supervised	Small labeled + large unlabeled dataset	Image classification with few labels
Reinforcement	Agent learns via rewards and penalties	Game AI, robotics, recommendation
Self-supervised	Model generates its own training labels	GPT language models, BERT

SECTION 2

The 8-Step ML Pipeline

1	Problem Definition What variable do we predict? What metric defines success? Classification or regression?
2	Data Collection CSV files, databases, APIs, web scraping — quality matters more than quantity
3	EDA & Cleaning Understand distributions, find nulls, duplicates, outliers, class imbalance
4	Feature Engineering Create, encode, scale, and select features that maximize predictive signal
5	Model Selection Choose algorithm family based on data size, type, and interpretability needs
6	Train & Validate Fit on training data; tune hyperparameters using cross-validation
7	Evaluate on Test Final performance check on held-out data — never touch test set early!
8	Deploy & Monitor Serve predictions via API; watch for data drift and performance degradation

Bias-Variance Tradeoff

The Fundamental ML Problem

High Bias (Underfitting): Model too simple → fails to capture patterns → bad on BOTH train and test
 High Variance (Overfitting): Model too complex → memorises noise → great train, terrible test
 Sweet Spot: Complex enough to learn real patterns, simple enough to generalise to new data

	Signs	How to Fix
	Low train acc + Low test acc	More features, more complex model, less regularisation
	High train acc + Low test acc	More data, regularisation (L1/L2), dropout, simpler model
	High train acc ≈ High test acc	Maintain this balance — this is the goal!

SECTION 4

Train / Validation / Test Split

Always split data **before** any preprocessing. Fit transformers (scalers, encoders) on **training data only**, then apply the same transform to validation and test sets. This prevents data leakage.

	Typical Size	Purpose
	60–80%	Model learns patterns from this data
	10–20%	Hyperparameter tuning and model selection
	10–20%	Final unbiased performance estimate — use ONCE

```
from sklearn.model_selection import train_test_split # 80/20 split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y # stratify keeps class ratio )
# 5-fold cross-validation (better estimate of performance)
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='f1')
print(f'CV F1: {scores.mean():.4f} +/- {scores.std():.4f}')
```

SECTION 5

Key Evaluation Metrics

	Formula	When to Use
	$(TP+TN)/Total$	Only for balanced classes
	$TP/(TP+FP)$	When false positives are costly (spam)
	$TP/(TP+FN)$	When false negatives are costly (fraud, disease)
	$2*P*R/(P+R)$	Imbalanced datasets — balances P and R
	Area under ROC curve	Overall model ranking ability [0,1]
	$\sqrt{\text{mean}((y-yp)^2)}$	Regression — penalises large errors
	$1 - SS_{res}/SS_{tot}$	Regression — proportion of variance explained

■ PRACTICE Q & A

Q1. What is the key difference between supervised and unsupervised learning?

A: Supervised uses labeled data (known outputs) to learn a mapping. Unsupervised finds hidden patterns in data without any labels.

Q2. Your model gets 99% train accuracy but 61% test accuracy. What is wrong and how do you fix it?

A: Overfitting — the model memorised training data but fails to generalise. Fix: add regularisation, collect more data, use a simpler model, or apply dropout.

Q3. Why should you NEVER fit a scaler on the entire dataset before splitting?

A: Data leakage — the scaler would see test data statistics and encode them into training. Always fit on train only, then transform both train and test.

Q4. What is cross-validation and why is it better than a single validation split?

A: CV splits data into K folds and trains/validates K times, using each fold as validation once. Gives a more reliable performance estimate with less variance.

Q5. When would you choose Recall over Precision as your key metric?

A: When missing a positive (false negative) is more costly than a false alarm — e.g. cancer screening, fraud detection. You want to catch every real case.

■ ML Fundamentals Cheat Sheet	
Supervised	Input + Label → learn mapping
Unsupervised	Input only → find structure
Overfitting	Great train, bad test – too complex
Underfitting	Bad train AND test – too simple
Bias	Error from wrong assumptions
Variance	Error from sensitivity to noise
<code>train_test_split(X,y,0.2)</code>	80/20 split
<code>cross_val_score(m,X,y,cv=5)</code>	5-fold cross-validation
Precision = $TP/(TP+FP)$	Avoid false alarms
Recall = $TP/(TP+FN)$	Catch all positives
F1 = $2 \cdot P \cdot R / (P+R)$	Balance precision and recall
AUC-ROC	Overall discrimination power [0,1]